Assumptions for this Course

- Assumption:
  - **You have taken a software engineering course at undergraduate level preferably object-oriented**
  - **Experience in object-oriented programming**
- Beneficial:
  - **You have had practical experience with a large software system**
  - **You have already participated in a large software project**
  - **You have experienced major problems.**

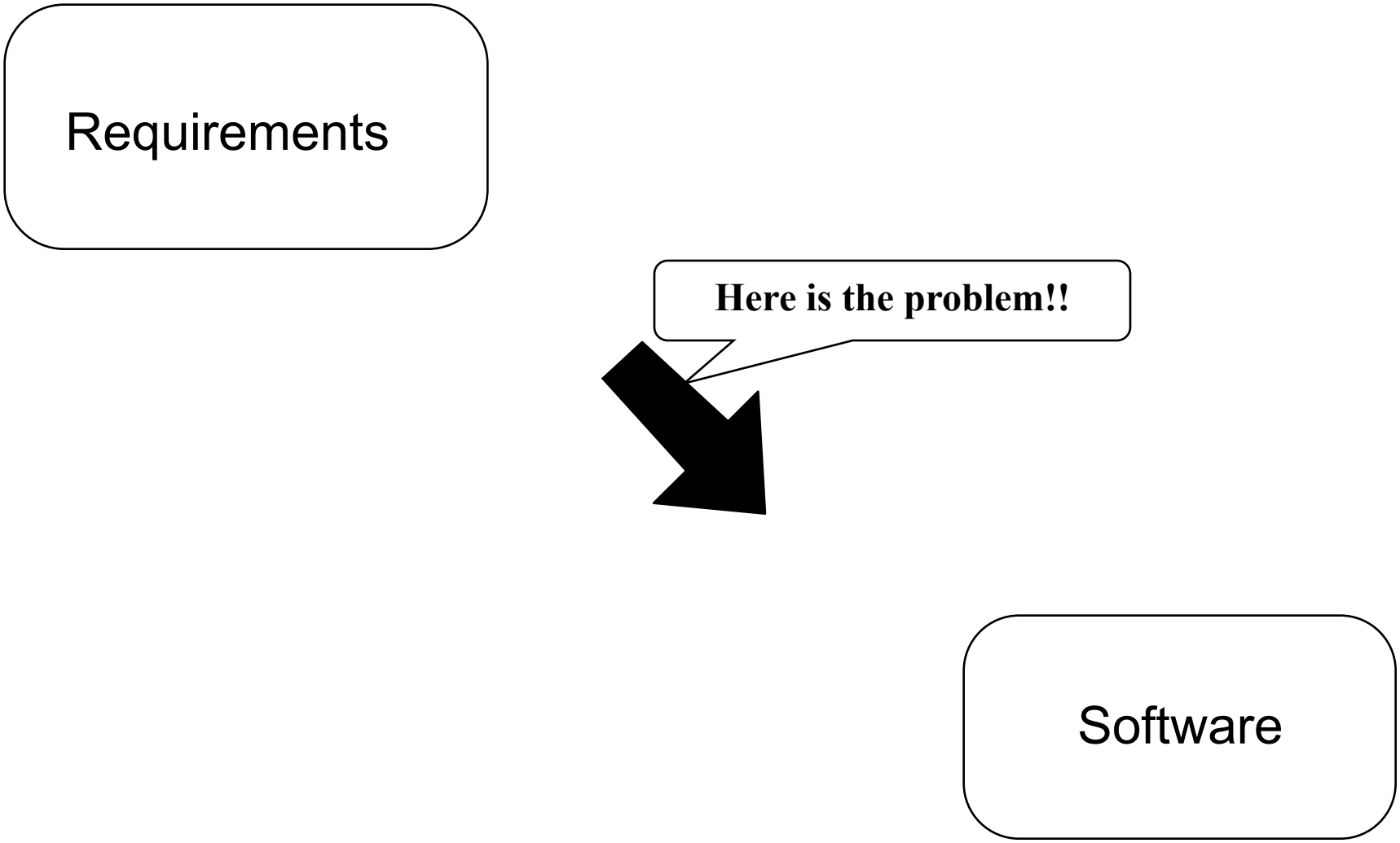
Objectives of the Course

- Understand how to
  - **produce a high quality software system within time while dealing with complexity and change**
- Acquire technical knowledge (main emphasis)
- Acquire managerial knowledge (through project)

Outline of Today's Lecture

- The development challenge
- Dealing with change
- Concepts: Abstraction, Modeling, Hierarchy
- Software Life Cycle Activities

Limitations of Non-engineered Software



Why is Software Development difficult?

- The problem is usually ambiguous
- The requirements are usually unclear and changing when they become clearer
- The problem domain (called application domain) is complex, and so is the solution domain
- The development process is difficult to manage
- Software offers extreme flexibility
- Invisibility

Software Development is more than just writing Code

- It is problem solving
  - **Understanding a problem**
  - **Proposing a solution and plan**
  - **Engineering a system based on the proposed solution using a *good* design**
- It is about dealing with complexity
  - **Creating abstractions and models**
  - **Notations for abstractions**
- It is knowledge management
  - **Elicitation, analysis, design, validation of the system and the solution process**
- It is rationale management
  - **Making the design and development decisions explicit to all stakeholders involved.**

Software Engineering: A Problem Solving Activity

- **Analysis**: Understand the nature of the problem and break the problem into pieces
- **Synthesis**: Put the pieces together into a large structure

For problem solving we use

- Techniques:
  - **Formal procedures for producing results using some well-defined notation**
- Methodologies:
  - **Collection of techniques applied across software development and unified by a philosophical approach**
- Tools:
  - **Instrument or automated systems to accomplish a technique (e.g., IDE, CASE)**

Computer Science vs. Engineering

- Computer Scientist
  - **Assumes techniques and tools have to be developed.**
  - **Proves theorems about algorithms, designs languages, defines knowledge representation schemes**
  - **Has infinite time…**
- Engineer
  - **Develops a solution for a problem formulated by a client**
  - **Uses computers & languages, techniques and tools**
- Software Engineer
  - **Works in multiple application domains**
  - **Has only 3 months...**
  - **…while changes occur in the problem formulation (requirements) and also in the available technology.**

Software Engineering: A Working Definition

Software Engineering is a collection of techniques, methodologies and tools that help with the production of

A high quality ***software*** **system developed with a given** *budget*
before a given *deadline*
while *change* occurs

Challenge: Dealing with complexity and change

Factors affecting the quality of a software system

- **Complexity:**
  - The system is so complex that no single software engineer can understand it anymore
  - The introduction of one bug fix causes another bug

- **Change:**
  - The “Entropy” of a software system increases with each change: Each implemented change erodes the structure of the system which makes the next change even more expensive.
  - As time goes on, the cost to implement a change will be too high, and the system will then be unable to support its intended task. This is true of all systems, independent of their application domain or technological base.

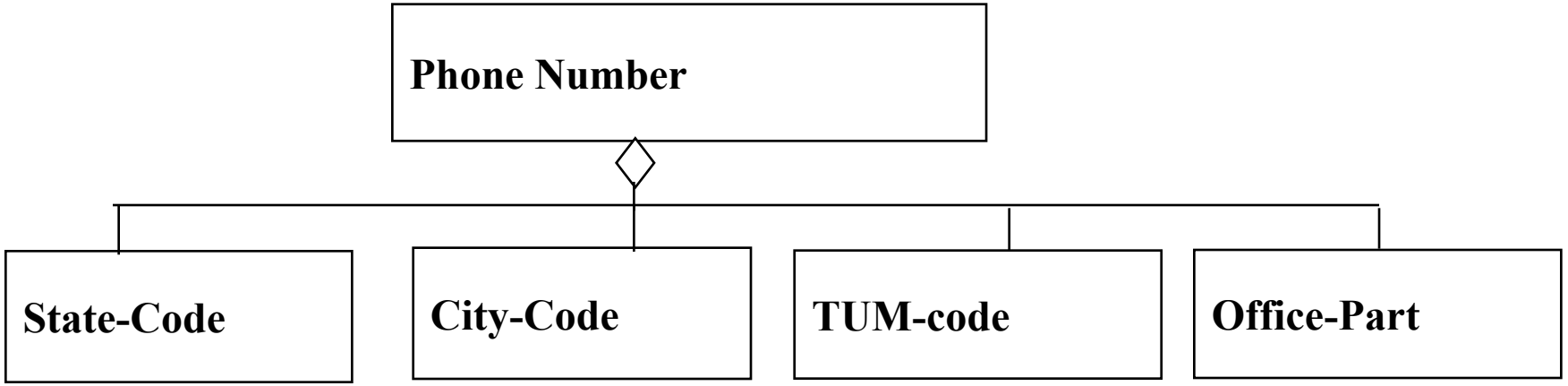
Dealing with Complexity – three ways

1. Abstraction
2. Decomposition
3. Hierarchy

Abstraction

- Complex systems are hard to understand
  - The 7 +- 2 phenomena
    - Our short term memory cannot store more than 7+-2 pieces at the same time -> limitation of the brain
    - Phone Number: 498928918204

- Chunking:
    - Group collection of objects to reduce complexity
    - 4 chunks:
      - State-code, city-code, Unv.-code, Office-Part



Abstraction

- Abstraction allows us to ignore unessential details
- Two definitions for abstraction:
  - Abstraction is a *thought process* where ideas are distanced from objects
    - **Abstraction as activity**
  - Abstraction is the *resulting idea* of a thought process where an idea has been distanced from an object
    - **Abstraction as entity**
- Ideas can be expressed by models



Models are used to provide abstractions

- System Model:
  - **Object Model: What is the structure of the system? What are the objects and how are they related?**
  - **Functional model: What are the functions of the system? How is data flowing through the system?**
  - **Dynamic model: How does the system react to external events? How is the event flow in the system ?**
- Task Model:
  - **PERT Chart: What are the dependencies between the tasks?**
  - **Schedule: How can this be done within the time limit?**
  - **Org Chart: What are the roles in the project or organization?**
- Issues Model:
  - **What are the open and closed issues? What constraints were posed by the client? What resolutions were made?**

Model-based software Engineering: Code is a derivation of object model

Problem Statement : A stock exchange lists many companies. Each company is identified by a ticker symbol

Analysis phase results in object model (UML Class Diagram):



Implementation phase results in code

```
public class StockExchange
{

 public Vector m_Company = new Vector();

};


public class Company
{

 public int m_tickerSymbol
 public Vector m_StockExchange = new Vector();

};
```

A good software engineer writes as little code as possible

2. Decomposition

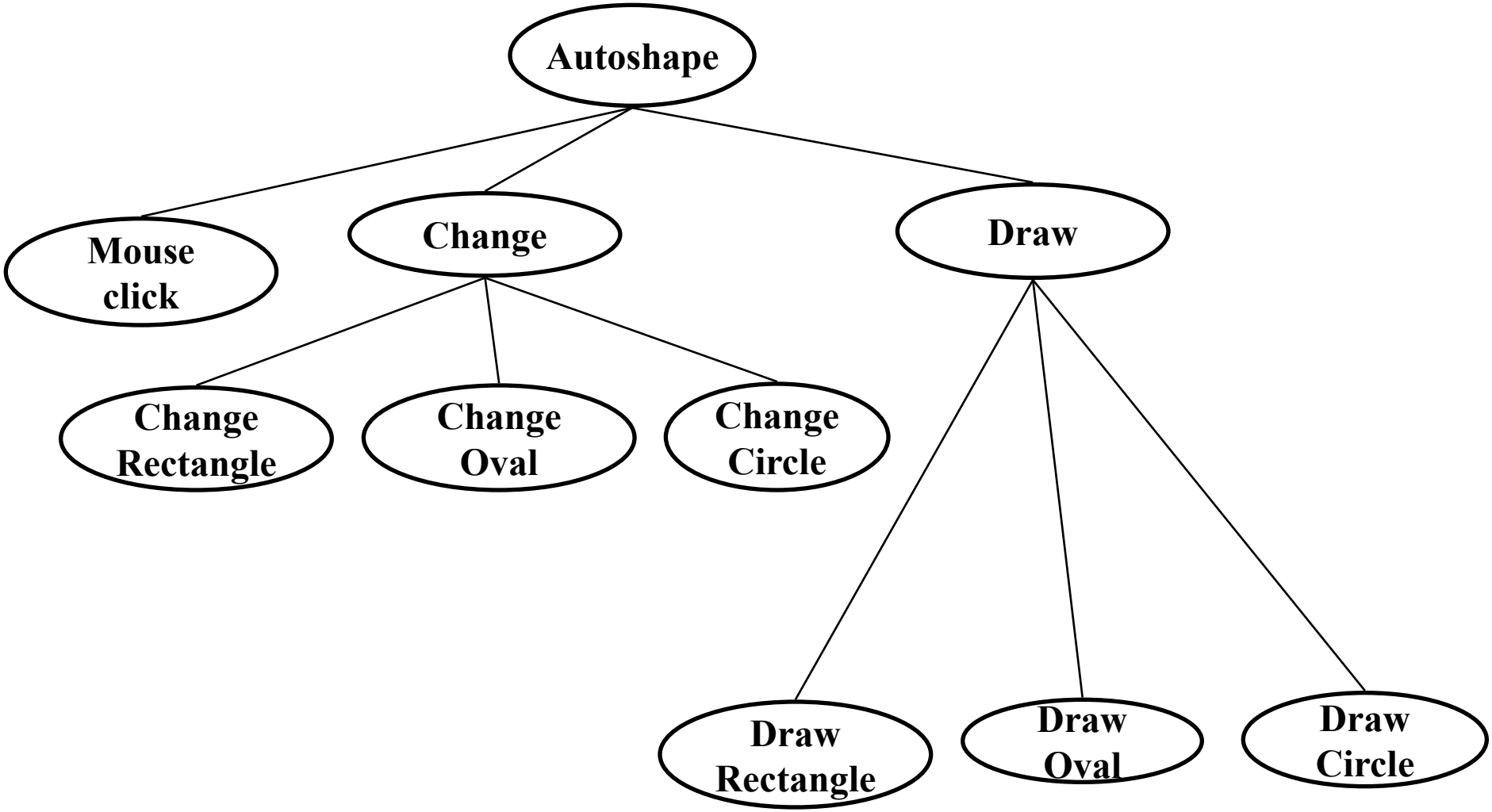
- A technique used to master complexity ("divide and conquer")
- Functional decomposition
  - **The system is decomposed into modules**
  - **Each module is a major processing step (function) in the application domain**
  - **Modules can be decomposed into smaller modules**
- Object-oriented decomposition
  - **The system is decomposed into classes ("objects")**
  - **Each class is a major abstraction in the application domain**
  - **Classes can be decomposed into smaller classes**

Which decomposition is the right one?

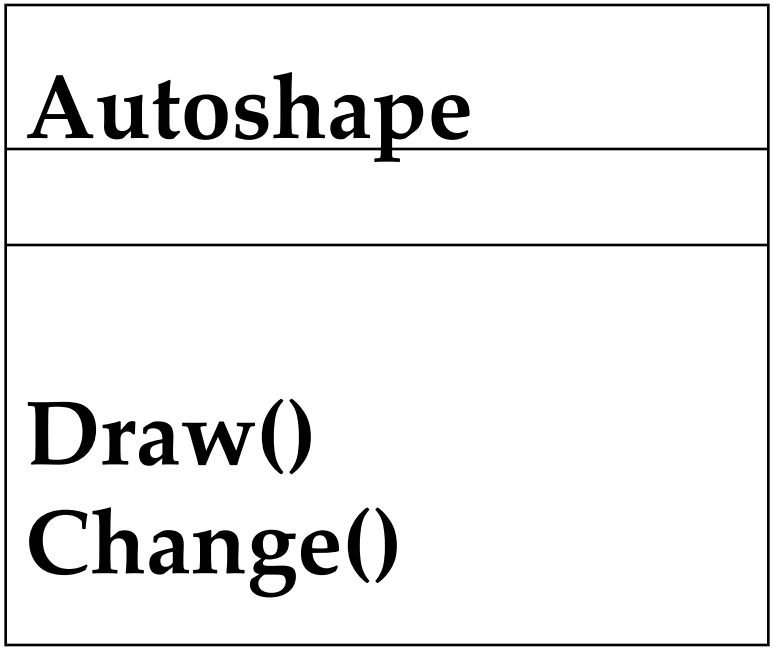
Functional Decomposition

- Functionality is spread all over the system
- Maintainer must understand the whole system to make a single change to the system
- Consequence:
  - **Codes are hard to understand**
  - **Code that is complex and impossible to maintain**
  - **User interface is often awkward and non-intuitive**

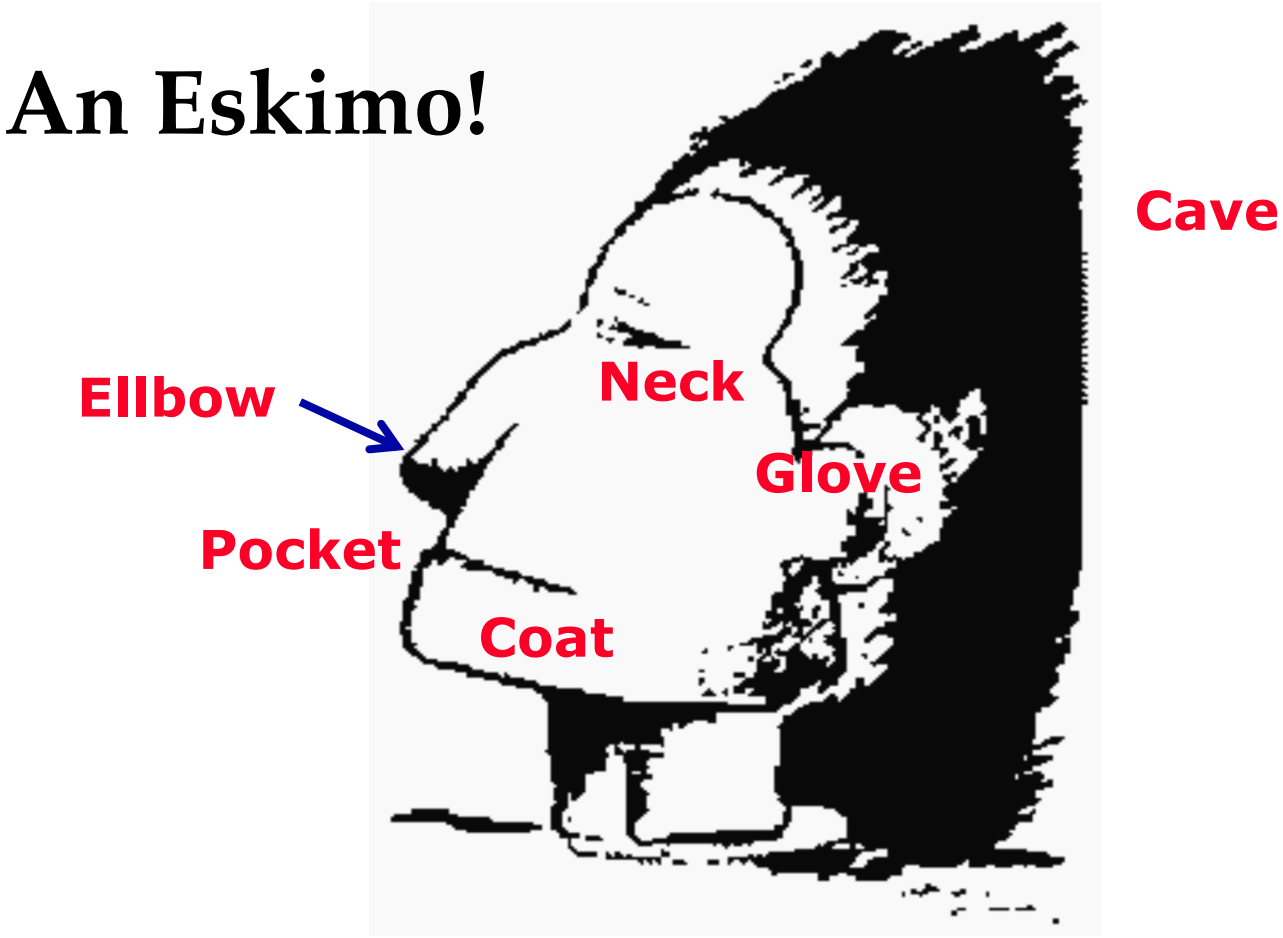
Functional Decomposition: Autoshape



Object-Oriented View

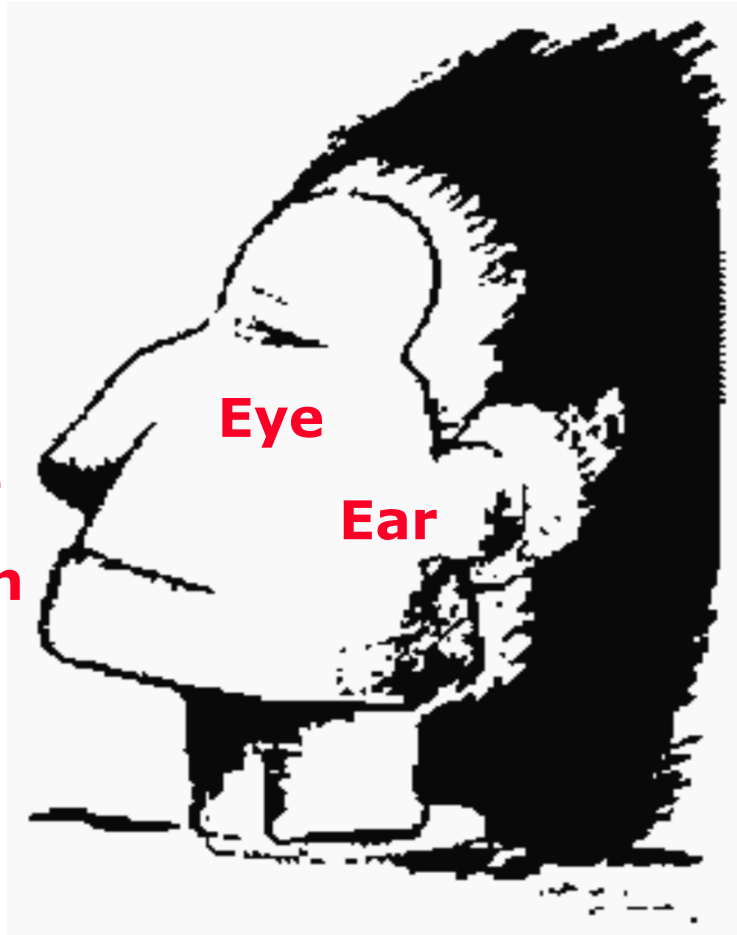


What is This?



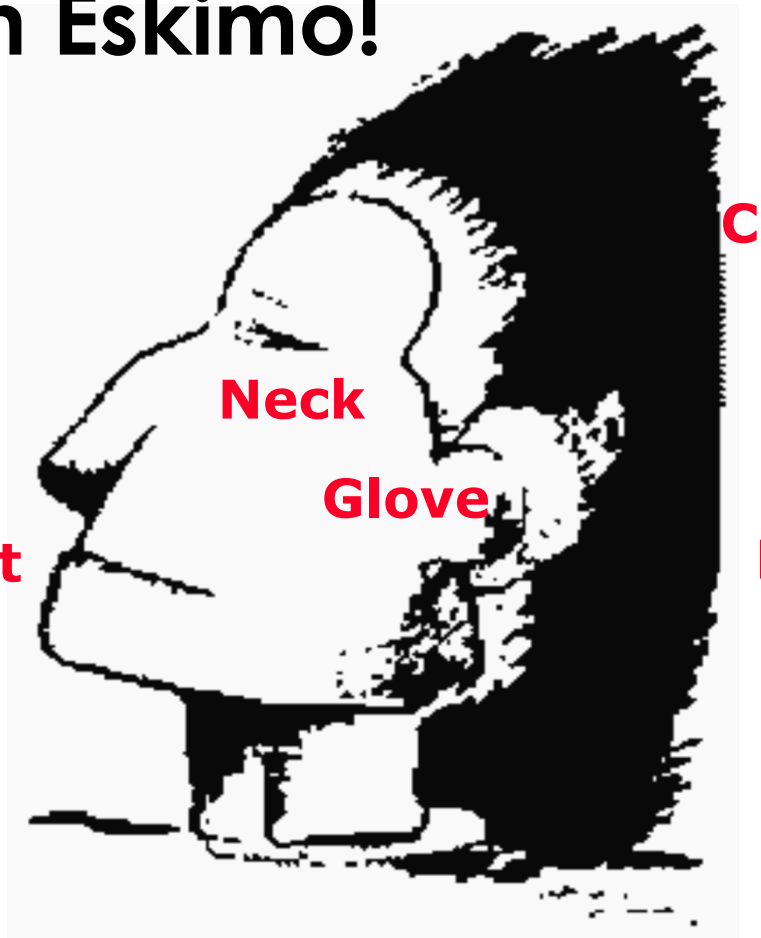
What is This?...

A Face!

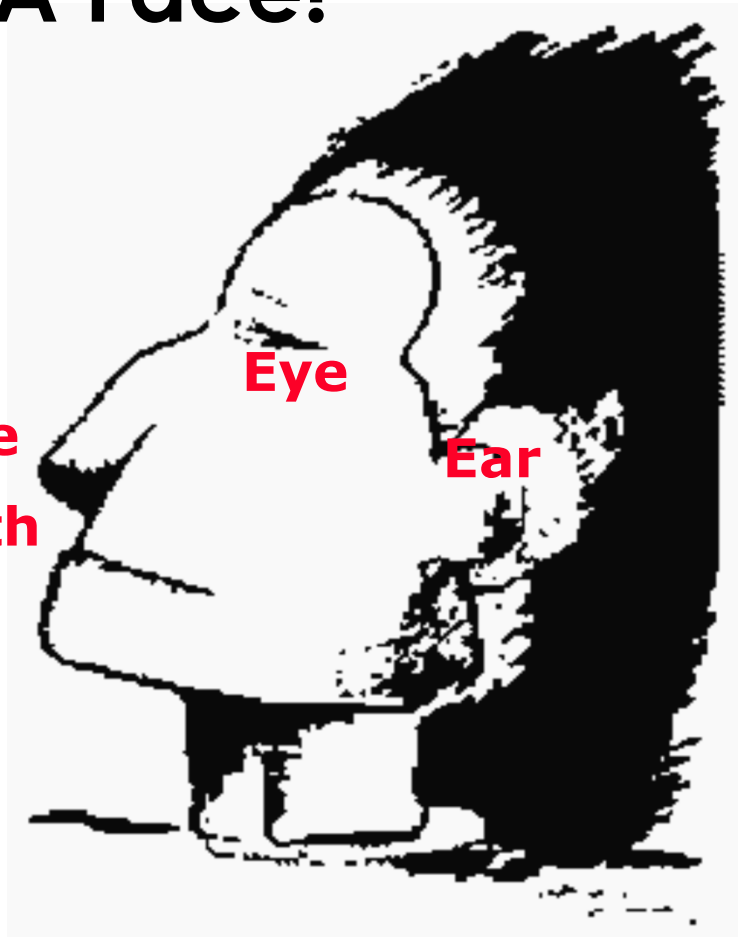


What is This?...

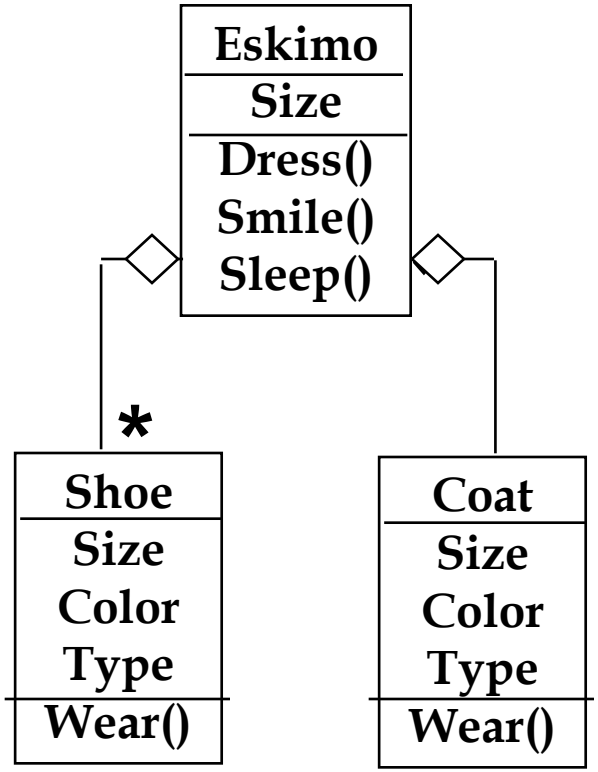
An Eskimo!



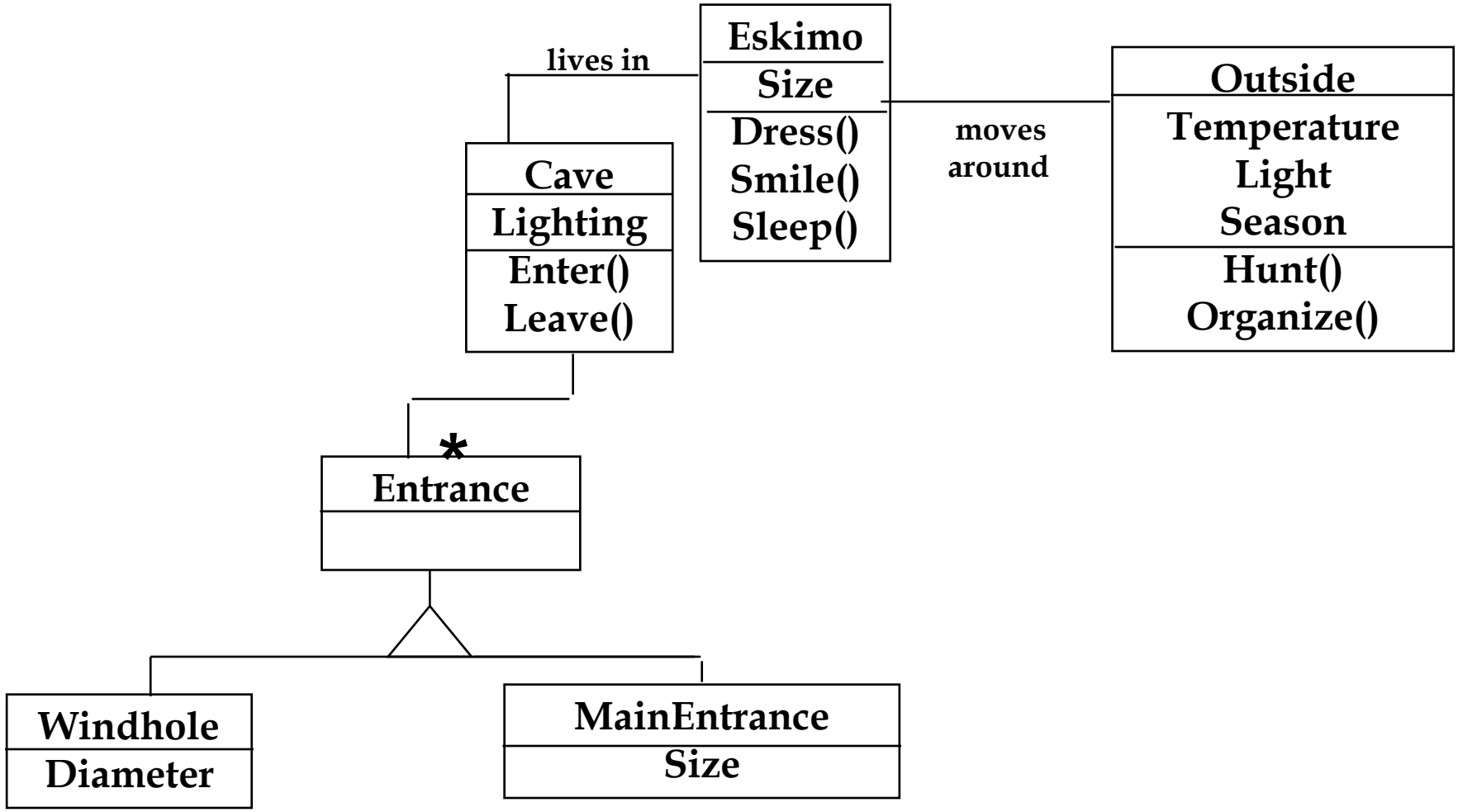
A Face!



Model of an Eskimo

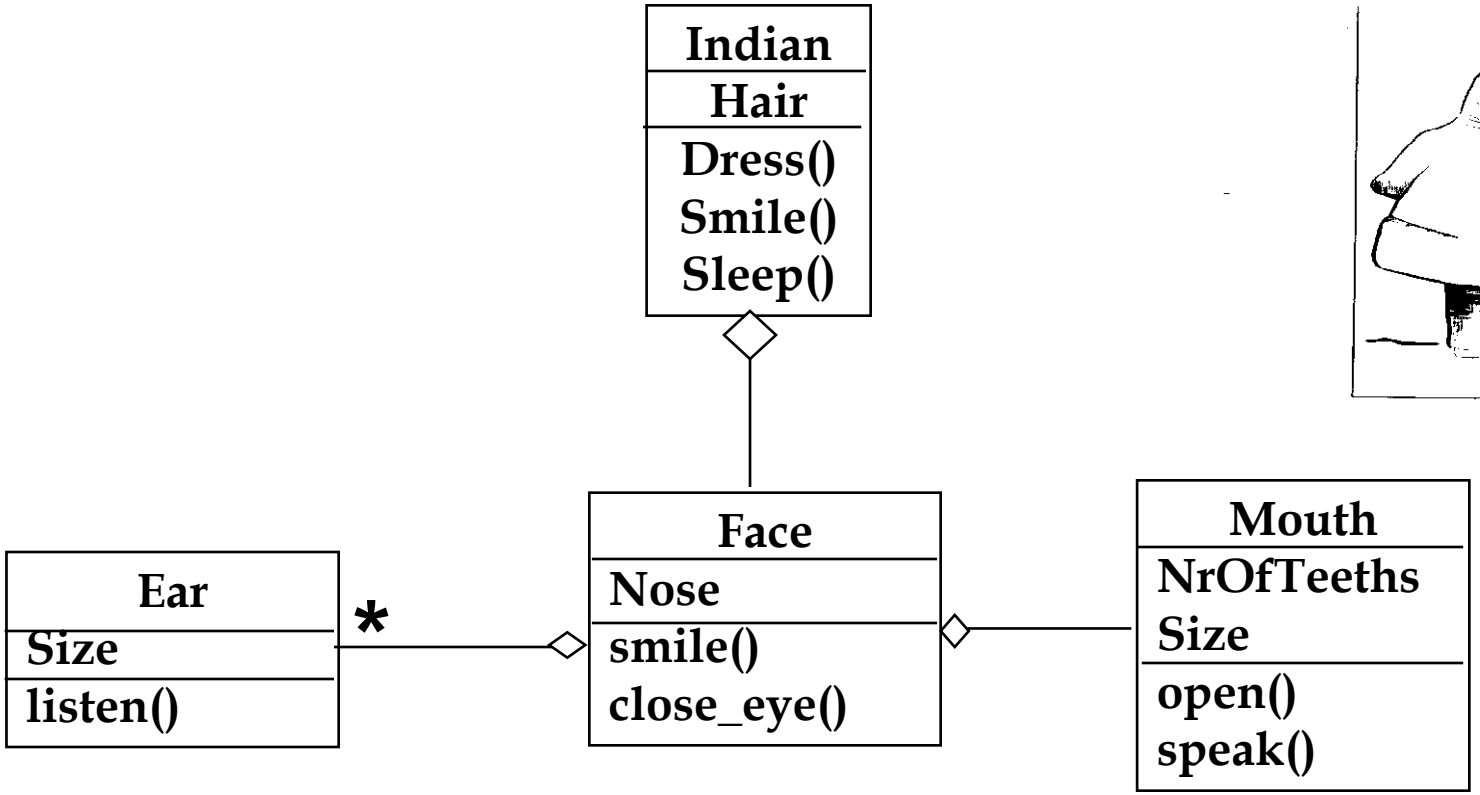


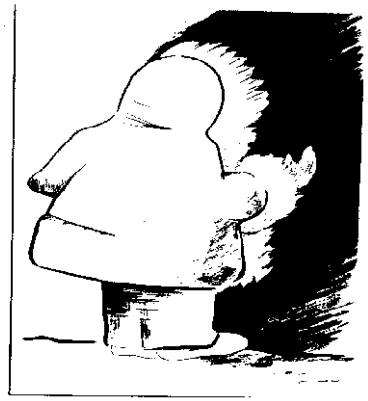
Iterative Modeling then leads to ....



but is it the right model?

Alternative Model: The Head of a Person

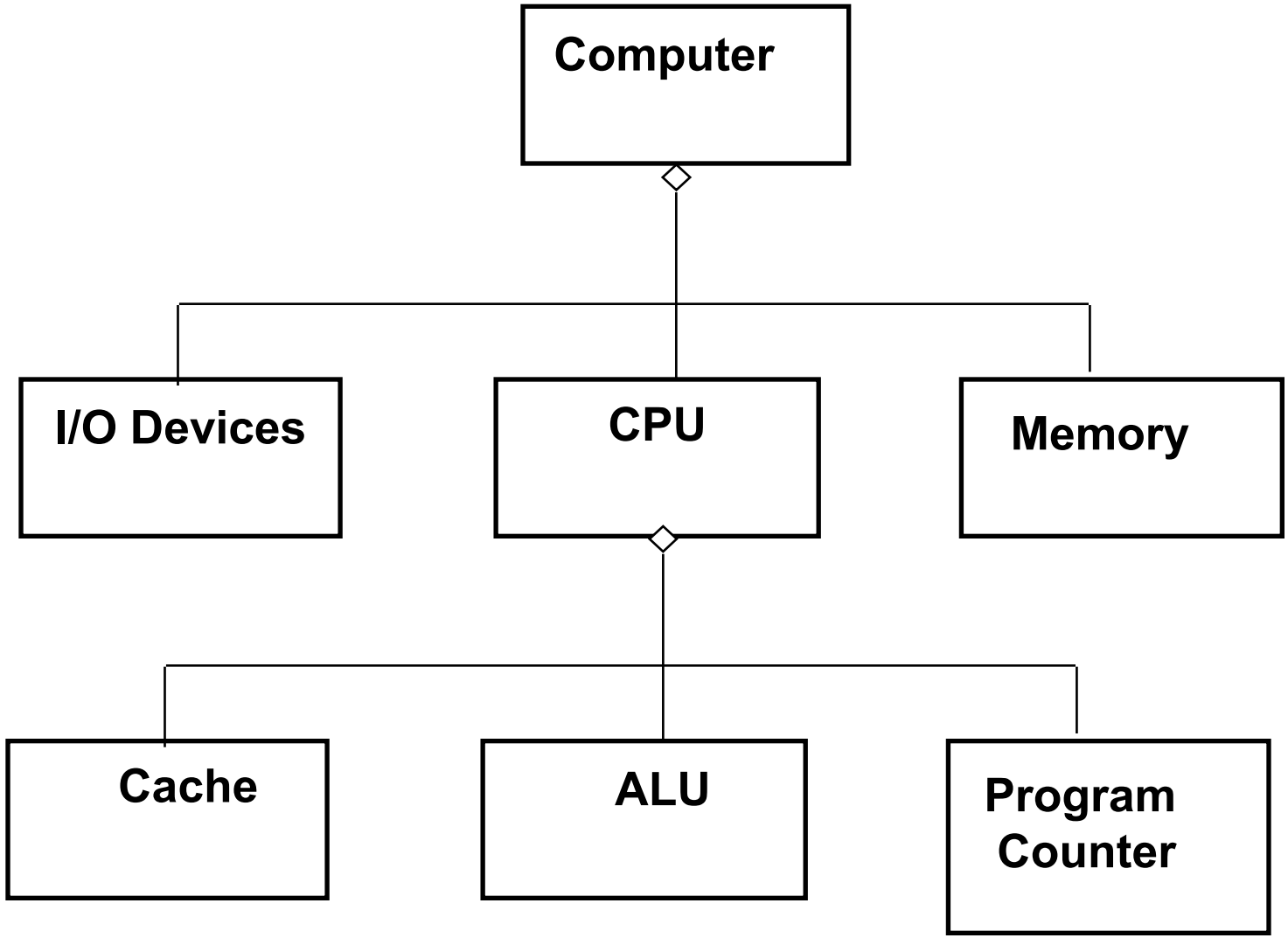




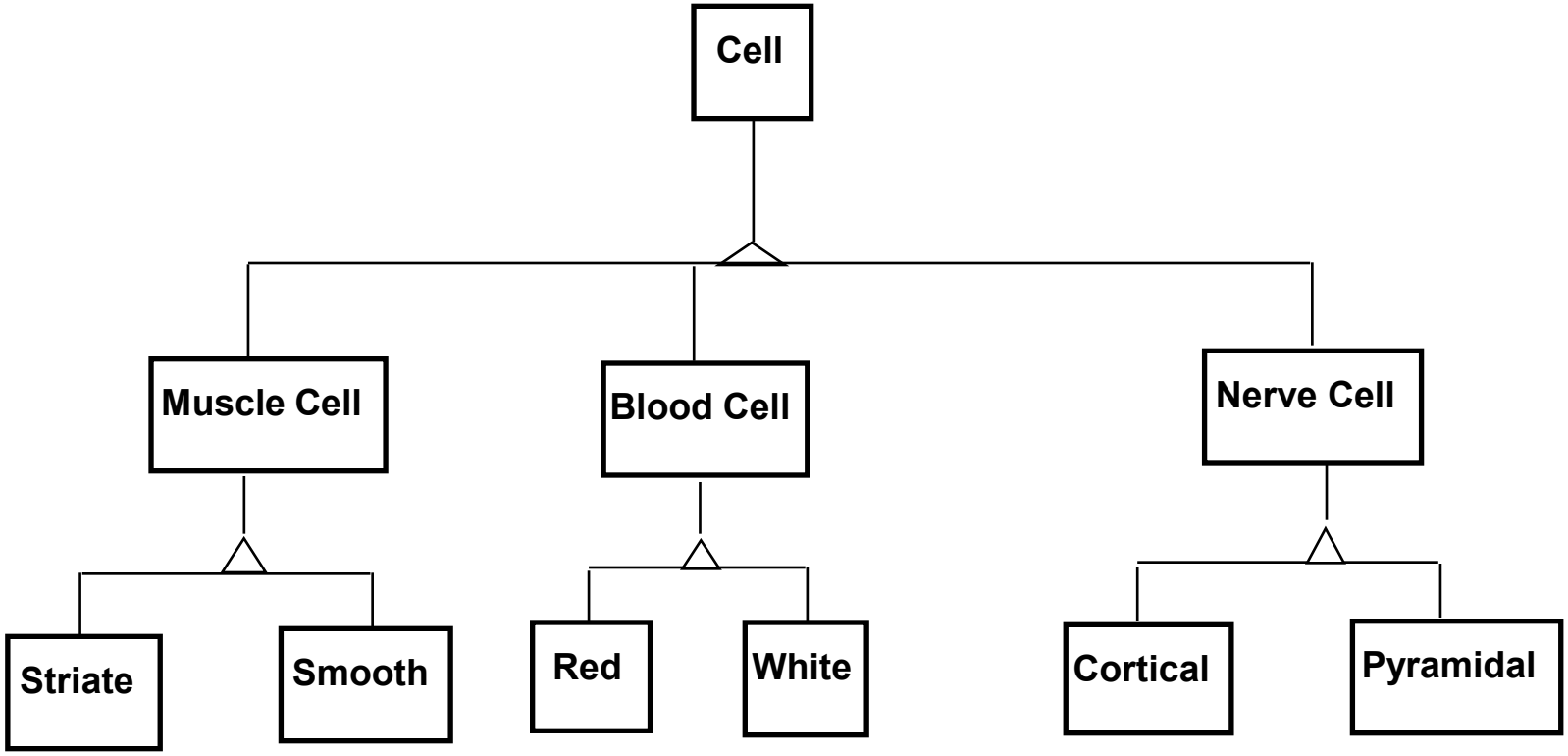
3. Hierarchy

- We got abstractions and decomposition
  - **This leads us to chunks (classes, objects) which we view with object model**
- Another way to deal with complexity is to provide simple relationships between the chunks
- One of the most important relationships is hierarchy
- 2 important hierarchies
  - **"Part of" hierarchy**
  - **"Is-kind-of" hierarchy**

Part of Hierarchy



Is-Kind-of Hierarchy (Taxonomy)

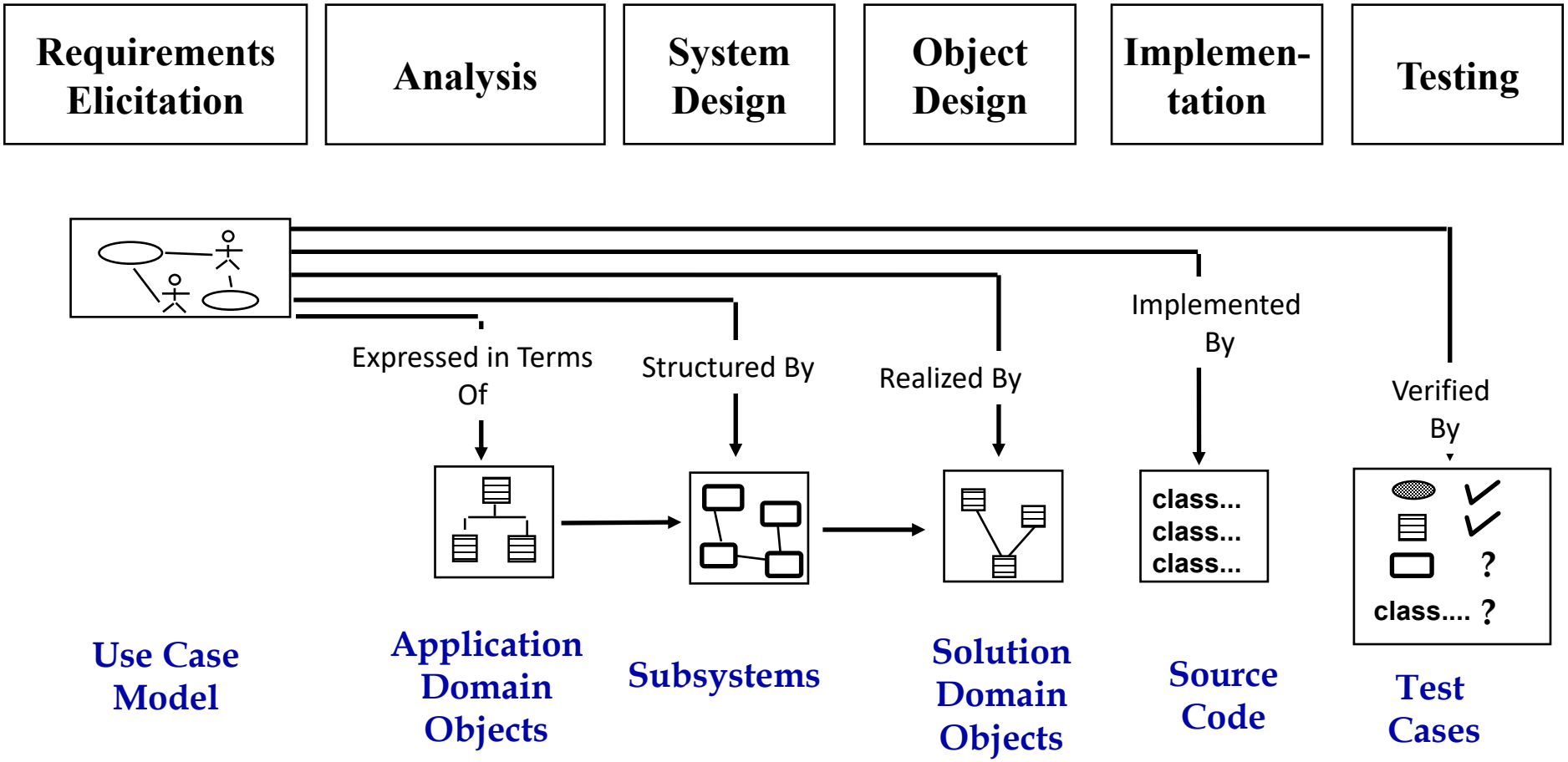


Recap

- Three ways to deal with complexity:
  - **Abstraction**
  - **Decomposition**
  - **Hierarchy**
- Object-oriented decomposition is a good methodology
  - **Unfortunately, depending on the purpose of the system, different objects can be found**
- How can we do it right?
  - **Many different possibilities**
  - **Our current approach: Start with a description of the functionality (Use case model), then proceed to the object model**
  - **This leads us to the software lifecycle**

Software Lifecycle Activities

...and their models



Summary

- Software engineering is a problem solving activity
  - **Developing quality software for a complex problem within a limited time while things are changing**
- There are many ways to deal with complexity
  - **Modeling, decomposition, abstraction, hierarchy**
- Many ways to deal with change
  - **Tailor the software lifecycle to deal with changing project conditions**
  - **Use a nonlinear software lifecycle to deal with changing requirements or changing technology**
  - **Provide configuration management to deal with changing entities**